

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

Code repository: <https://github.com/Snigdha-clo/Deep-Learning/tree/main/-Multi-Layer-Perceptron>

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Output:

Training data shape: (60000, 28, 28)

Training labels shape: (60000,)

Testing data shape: (10000, 28, 28)

Testing labels shape: (10000,)

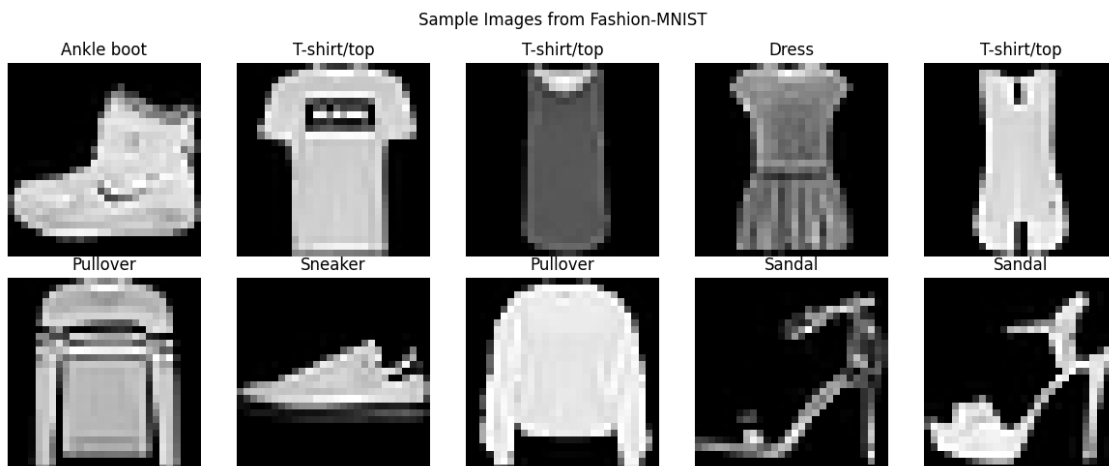


Figure 1: Sample images from the Fashion-MNIST training set

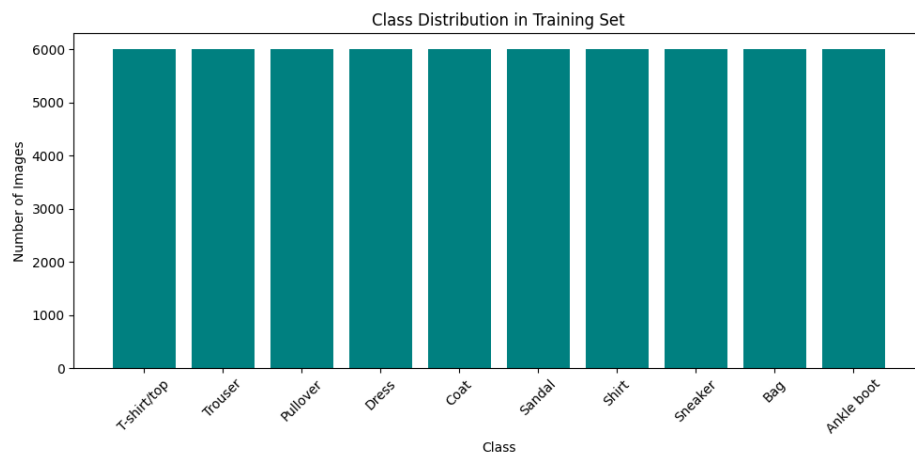


Figure 2: Class distribution in the training set

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Output:

Shape before preprocessing:

x_train: (60000, 28, 28) | x_test: (10000, 28, 28)

Shape after preprocessing:

x_train_flat: (60000, 784) | x_test_flat: (10000, 784)

y_train_ohe: (60000, 10) | y_test_ohe: (10000, 10)

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Output:

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Output:

Epoch	1/20	accuracy: 0.9381	loss: 0.1618	val_accuracy: 0.8866	val_loss: 0.4048
Epoch	2/20	accuracy: 0.9399	loss: 0.1566	val_accuracy: 0.8847	val_loss: 0.4254
Epoch	3/20	accuracy: 0.9420	loss: 0.1505	val_accuracy: 0.8861	val_loss: 0.4484
Epoch	4/20	accuracy: 0.9444	loss: 0.1464	val_accuracy: 0.8870	val_loss: 0.4549
Epoch	5/20	accuracy: 0.9450	loss: 0.1446	val_accuracy: 0.8802	val_loss: 0.4739
Epoch	6/20	accuracy: 0.9474	loss: 0.1387	val_accuracy: 0.8813	val_loss: 0.4800
Epoch	7/20	accuracy: 0.9474	loss: 0.1364	val_accuracy: 0.8808	val_loss: 0.5172
Epoch	8/20	accuracy: 0.9491	loss: 0.1324	val_accuracy: 0.8800	val_loss: 0.5154
Epoch	9/20	accuracy: 0.9503	loss: 0.1313	val_accuracy: 0.8827	val_loss: 0.4747
Epoch	10/20	accuracy: 0.9501	loss: 0.1271	val_accuracy: 0.8827	val_loss: 0.5169

Epoch 11/20	accuracy: 0.9539	loss: 0.1220	val_accuracy: 0.8801	val_loss: 0.5555
Epoch 12/20	accuracy: 0.9534	loss: 0.1226	val_accuracy: 0.8803	val_loss: 0.5537
Epoch 13/20	accuracy: 0.9545	loss: 0.1178	val_accuracy: 0.8882	val_loss: 0.5180
Epoch 14/20	accuracy: 0.9546	loss: 0.1193	val_accuracy: 0.8833	val_loss: 0.5669
Epoch 15/20	accuracy: 0.9572	loss: 0.1123	val_accuracy: 0.8824	val_loss: 0.5923
Epoch 16/20	accuracy: 0.9581	loss: 0.1083	val_accuracy: 0.8812	val_loss: 0.6041
Epoch 17/20	accuracy: 0.9555	loss: 0.1168	val_accuracy: 0.8763	val_loss: 0.6053
Epoch 18/20	accuracy: 0.9578	loss: 0.1108	val_accuracy: 0.8842	val_loss: 0.6062
Epoch 19/20	accuracy: 0.9601	loss: 0.1051	val_accuracy: 0.8768	val_loss: 0.6462
Epoch 20/20	accuracy: 0.9595	loss: 0.1073	val_accuracy: 0.8818	val_loss: 0.6170

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Output:

Accuracy: 0.8744

Precision: 0.8795

Recall: 0.8744

F1-score: 0.8754

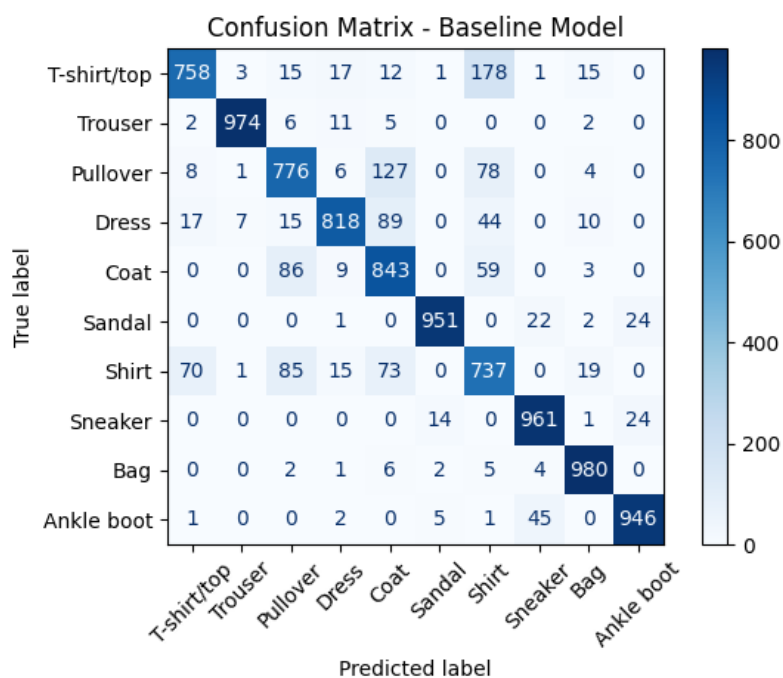


Figure 3: Confusion matrix – baseline model

Classification Report:

	precision	recall	f1-score	support
T-shirt/top	0.89	0.76	0.82	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.79	0.78	0.78	1000
Dress	0.93	0.82	0.87	1000
Coat	0.73	0.84	0.78	1000
Sandal	0.98	0.95	0.96	1000
Shirt	0.67	0.74	0.70	1000
Sneaker	0.93	0.96	0.95	1000
Bag	0.95	0.98	0.96	1000

Ankle boot	0.95	0.95	0.95	1000
accuracy			0.87	10000
macro avg	0.88	0.87	0.88	10000
weighted avg	0.88	0.87	0.88	10000

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

Output – Randomized Search:

Fitting 2 folds for each of 5 candidates, totalling 10 fits

Best Score: 0.7775

Best Params: {'optimizer': 'sgd', 'neurons': 128, 'learning_rate': 0.1,
'hidden_layers': 2, 'dropout_rate': 0.0, 'activation': 'tanh',
'epochs': 10, 'batch_size': 128}

Output – Retraining with Best Hyperparameters:

Epoch	1/10	accuracy: 0.7775	loss: 0.6262	val_accuracy: 0.8267	val_loss: 0.4857
Epoch	2/10	accuracy: 0.8341	loss: 0.4589	val_accuracy: 0.8451	val_loss: 0.4365
Epoch	3/10	accuracy: 0.8486	loss: 0.4186	val_accuracy: 0.8536	val_loss: 0.4135
Epoch	4/10	accuracy: 0.8579	loss: 0.3934	val_accuracy: 0.8572	val_loss: 0.4001
Epoch	5/10	accuracy: 0.8643	loss: 0.3749	val_accuracy: 0.8583	val_loss: 0.3911
Epoch	6/10	accuracy: 0.8693	loss: 0.3602	val_accuracy: 0.8590	val_loss: 0.3845
Epoch	7/10	accuracy: 0.8736	loss: 0.3478	val_accuracy: 0.8605	val_loss: 0.3794
Epoch	8/10	accuracy: 0.8779	loss: 0.3369	val_accuracy: 0.8612	val_loss: 0.3751
Epoch	9/10	accuracy: 0.8816	loss: 0.3272	val_accuracy: 0.8617	val_loss: 0.3713
Epoch	10/10	accuracy: 0.8843	loss: 0.3184	val_accuracy: 0.8631	val_loss: 0.3679

Optimized Accuracy: 0.8538
Optimized Precision: 0.8680
Optimized Recall: 0.8538

Optimized F1-score: 0.8561

Baseline vs Optimized Accuracy: 0.8744 -> 0.8538

8. Mandatory Plots

Generate the following plots.

- 1. Sample Images
 - 2. Class Distribution
 - 3. Training Accuracy vs Epoch
 - 4. Validation Accuracy vs Epoch
 - 5. Training Loss vs Epoch
 - 6. Validation Loss vs Epoch
 - 7. Confusion Matrix
 - 8. Hyperparameter Search Results
 - 9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

1. Sample Images *Inference:*

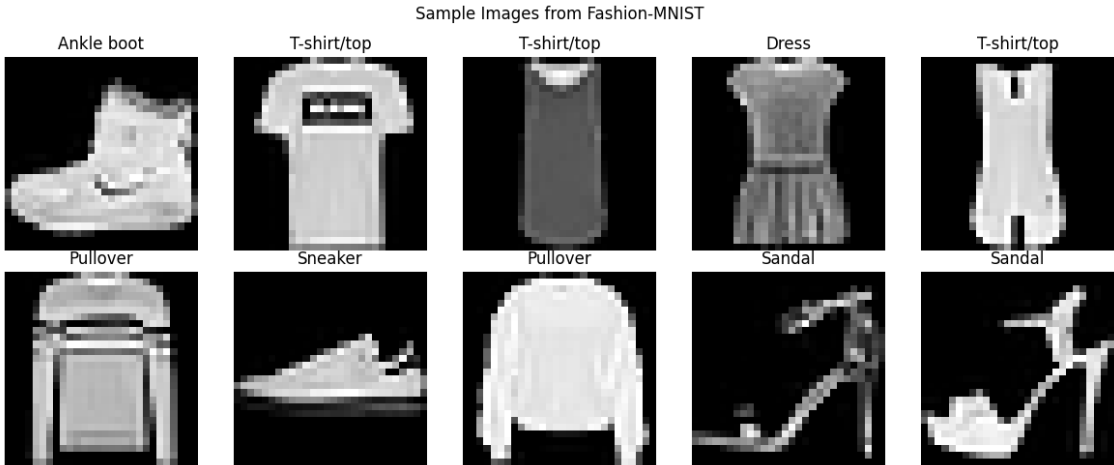


Figure 4: Sample images from Fashion-MNIST

2. Class Distribution *Inference:*



Figure 5: Class distribution across the 10 categories

3 & 4. Training and Validation Accuracy vs Epoch *Inference*:

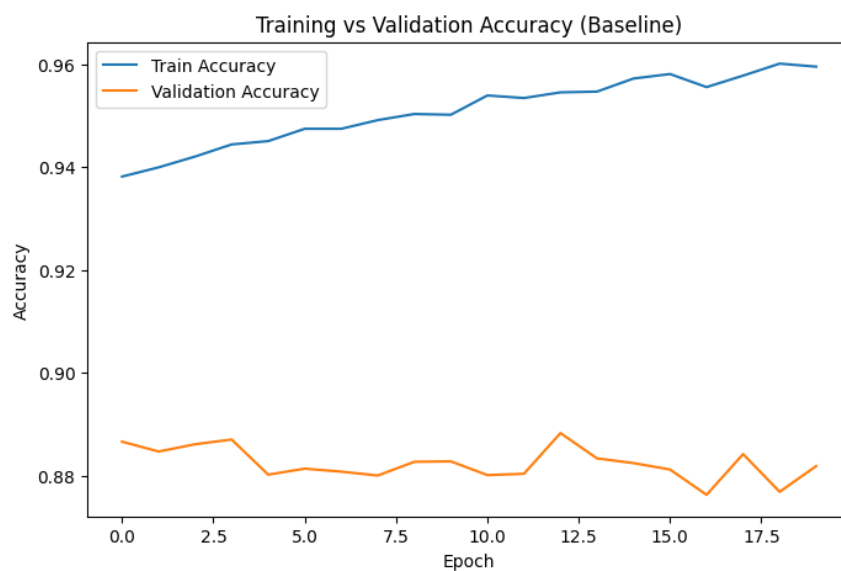


Figure 6: Training vs validation accuracy over epochs (baseline model)

5 & 6. Training and Validation Loss vs Epoch *Inference*:

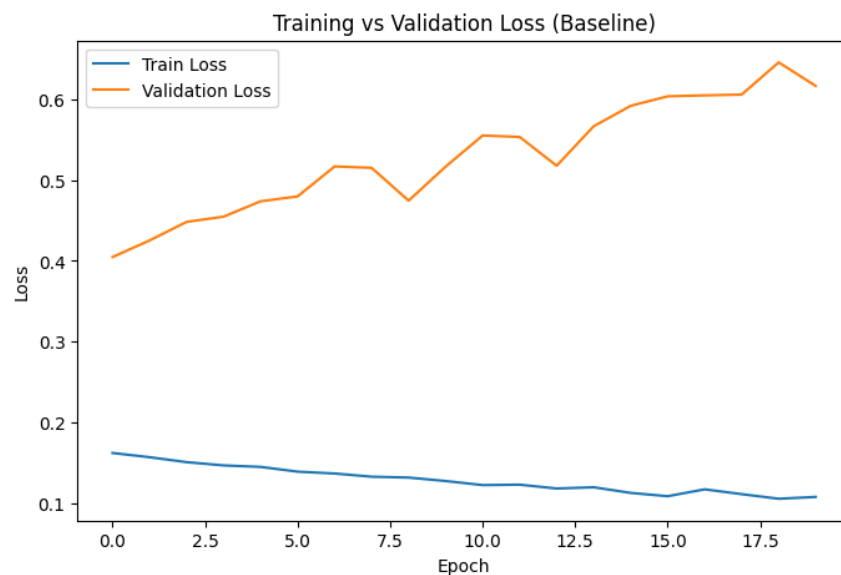


Figure 7: Training vs validation loss over epochs (baseline model)

7. Confusion Matrix *Inference*:



Figure 8: Confusion matrix – baseline model

8. Hyperparameter Search Results *Inference*:

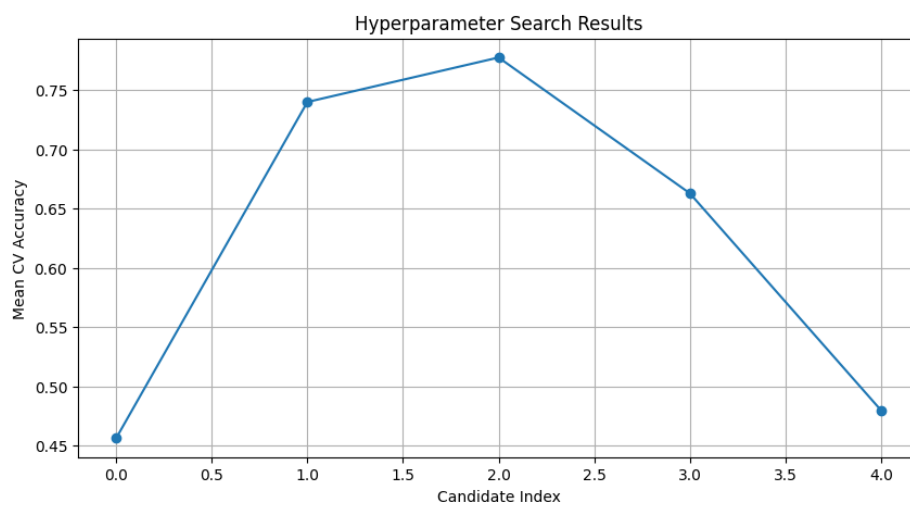


Figure 9: Mean cross-validation accuracy across candidate hyperparameter combinations

9. Best Model Accuracy Comparison *Inference*:

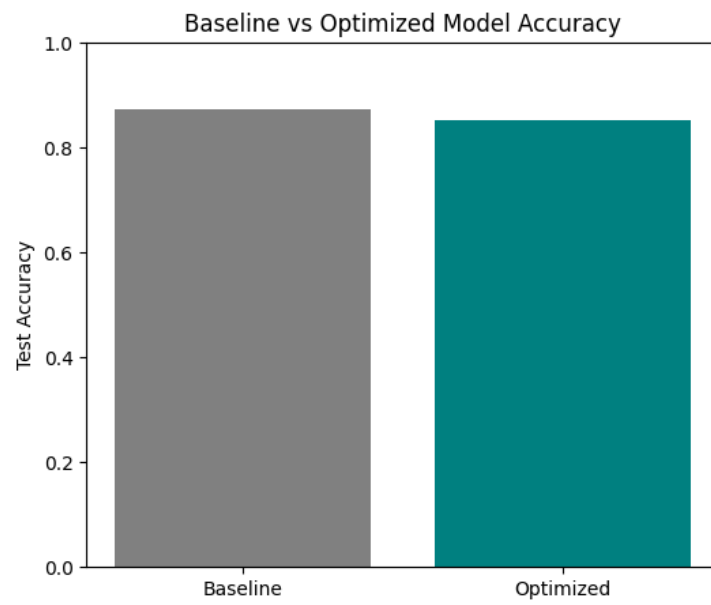


Figure 10: Baseline vs optimized model test accuracy

9. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	128
Learning Rate	0.1
Batch Size	128
Optimizer	SGD
Activation Function	Tanh
Epochs	10
Dropout	0.0
Cross-validation Accuracy	0.7775
Testing Accuracy	0.8538

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8744	0.8538
Precision	0.8795	0.8680
Recall	0.8744	0.8538
F1-score	0.8754	0.8561
Training Time	≈180s (20 epochs)	≈25s (10 epochs)

10. Discussion

Answer the following:

1. Which optimization method was used?
2. Which hyperparameters were selected?
3. Did optimization improve performance?
4. Which hyperparameter had the greatest impact?
5. Compare Grid Search and Randomized Search.
6. Recommend the best MLP configuration.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

A Appendix: Solving XOR with a Multi-Layer Perceptron

A.1 Motivation

Experiment 1's Single Layer Perceptron could not solve the XOR gate, since XOR is not linearly separable and a single perceptron can only represent one straight decision boundary. This appendix extends that experiment by training a Multi-Layer Perceptron (MLP) with one hidden layer on the XOR truth table, implemented from scratch in NumPy (weight initialization, sigmoid activation, forward propagation, and backpropagation), to show how a hidden layer resolves the XOR problem.

Architecture: 2 inputs $\rightarrow$ Dense(2, Sigmoid) hidden layer $\rightarrow$ Dense(1, Sigmoid) output layer, trained with batch gradient descent (learning rate 0.5) for up to 10,000 epochs.

A.2 Training Output

Weights were logged every 1000 epochs (plus the very first epoch) to track convergence:

```
Epoch    1 | Loss: 0.25164
  W1=[[0.544, -0.121], [0.714, 0.395]]  b1=[-0.002, 0.0]
  W2=[[-0.808], [0.952]]                b2=[0.002]

Epoch 1000 | Loss: 0.17623
  W1=[[0.745, 3.150], [0.938, 3.227]]  b1=[-0.700, -0.536]
  W2=[[-2.100], [3.098]]                b2=[-1.345]

Epoch 2000 | Loss: 0.00443
  W1=[[3.601, 5.546], [3.604, 5.561]]  b1=[-5.520, -2.291]
  W2=[[-7.758], [7.173]]                b2=[-3.230]

Epoch 3000 | Loss: 0.00178
  W1=[[3.979, 5.854], [3.982, 5.865]]  b1=[-6.110, -2.505]
  W2=[[-8.641], [7.995]]                b2=[-3.638]

Epoch 4000 | Loss: 0.00109
  W1=[[4.167, 6.013], [4.169, 6.022]]  b1=[-6.400, -2.608]
  W2=[[-9.103], [8.441]]                b2=[-3.862]

Epoch 5000 | Loss: 0.00078
  W1=[[4.290, 6.119], [4.292, 6.127]]  b1=[-6.589, -2.675]
  W2=[[-9.416], [8.747]]                b2=[-4.016]

Epoch 6000 | Loss: 0.00060
  W1=[[4.381, 6.198], [4.383, 6.205]]  b1=[-6.728, -2.723]
  W2=[[-9.652], [8.980]]                b2=[-4.134]

Epoch 7000 | Loss: 0.00049
  W1=[[4.452, 6.260], [4.454, 6.267]]  b1=[-6.837, -2.761]
  W2=[[-9.841], [9.167]]                b2=[-4.229]

Epoch 8000 | Loss: 0.00041
  W1=[[4.511, 6.312], [4.512, 6.319]]  b1=[-6.926, -2.792]
  W2=[[-9.999], [9.324]]                b2=[-4.309]
```

```

Epoch 9000 | Loss: 0.00035
  W1=[[4.560, 6.356], [4.562, 6.363]]  b1=[-7.002, -2.819]
  W2=[[-10.134], [9.459]]             b2=[-4.378]

Epoch 10000 | Loss: 0.00031
  W1=[[4.603, 6.395], [4.605, 6.401]]  b1=[-7.067, -2.841]
  W2=[[-10.252], [9.578]]             b2=[-4.438]

Final predictions: [0 1 1 0]
Expected:          [0 1 1 0]

```

Unlike the single-layer perceptron on XOR, the MLP's loss decreases toward zero and its final predictions exactly match the XOR truth table.

A.3 Why the MLP Succeeds Where the Single-Layer Perceptron Failed

The reason why a single layer perceptron is not enough to solve the problem is because it is capable of representing only one linear boundary ($\mathbf{w}^T \mathbf{x} + b = 0$) which is inherently inadequate for XOR, as both of its classes fall on opposite diagonals. By adding an additional layer of two neurons, the network can represent two different linear boundaries, while the output neuron would combine them using non-linear activation functions (sigmoid) into one curved decision area. This combination of linear boundaries is precisely the key to solving XOR problem by MLPs.

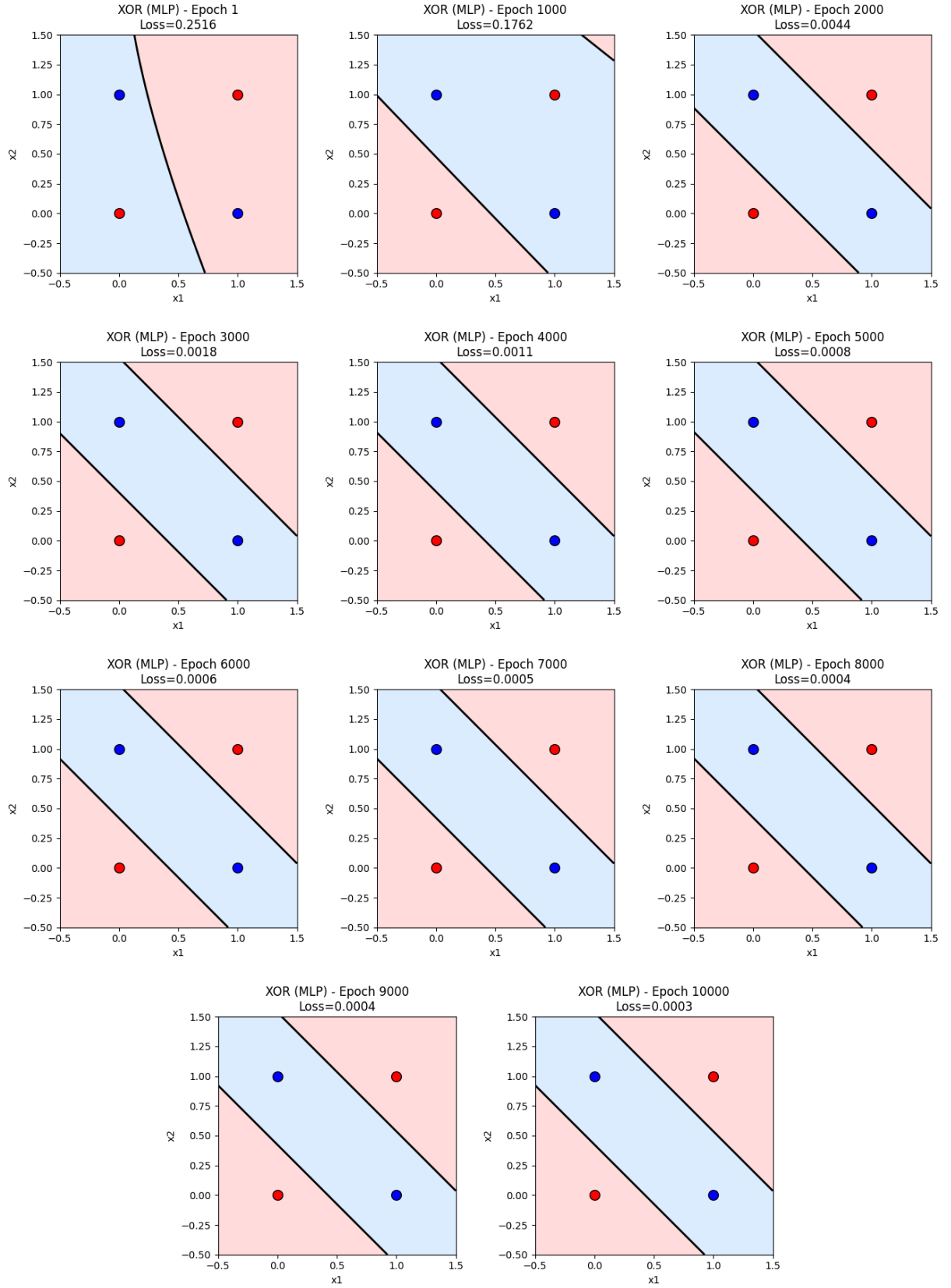


Figure 11: XOR decision boundary evolution using an MLP (checkpoints at epochs 1, 1000, 2000, ..., 10,000). The boundary warps from an arbitrary line into a curved region that correctly separates the two diagonal classes.